Chapter 3
The Fundamentals: Algorithms The Integers



Msc. Nguyen Quoc Thanh

Objectives

- Algorithms
- The Growth of Functions
- Complexity of Algorithms
- The Integers and Division
- Primes and Greatest Common Divisors (UCLN)
- Integers and Algorithms

3.1- Algorithms

An algorithm is a finite set of precise instructions for performing a computation or for solving a problem.

Specifying an algorithm: natural language/ pseudocode

Properties of an algorithm

- Input
- Output
- Definiteness - tính rõ ràng
- Correctness - tính chính xác
- Finiteness - tính hữu hạn
- Effectiveness - tính hiệu qủa
- Generality - tính tổng quát

3.1- Algorithms

Pseudocode for Finding the Maximum Element in a Finite Sequence

Procedure max (a_1,a_2,a_3,…,a_n: integers)

   max:=a_1

   for i:=2 **to** n

      if max < a_i **then** max:= a_i

{max is the largest element}

The Linear Search (tìm tuyến tính)

Procedure linear search (x: integer, $a_1,a_2,\ldots,a_n$:distinct integers)

i:=1

while ($i \leq n$ **and** $x \neq a_i$)

i:=i+1

if $i \leq n$ **then** location:= i

else location:=0

{location is the subscript of the term that equals x, or is 0 if x is not found}

The Binary Search (tìm nhị phân)

procedure binary search (x:integer, $a_1,a_2,\ldots,a_n$: increasing integers)

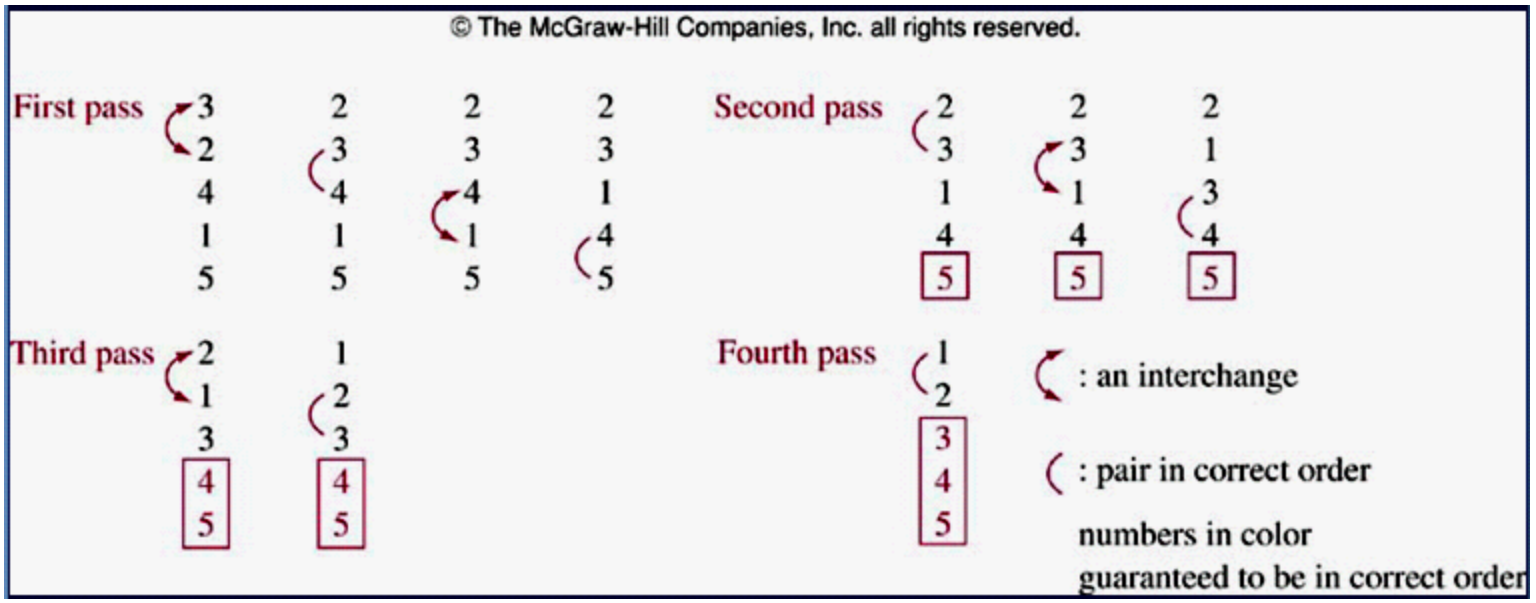
```
i:=1 { i is left endpoint of search interval}
j:=n { j is right endpoint of search interval}
while i<j
  begin
     m:= ⌊(i+j)/2⌋
     if  x>a_m then  i:=m+1
     else  j:= m
   end
   if  x=a_i then location := i
 else location:= 0
```

{location is the subscript of the term that equals x, or is 0 if x is not found}

Sorting (sắp xếp)

- Putting elements into a list in which the elements are in increasing order.
- There are some sorting algorithms
- Bubble sort
- Insertion sort
- Selection sort (exercise p. 178)
- Binary insertion sort (exercise p. 179)
- Shaker sort (exercise p.259)
- Merge sort and quick sort (section 4.4)
- Tournament sort (10.2)

Bubble Sort (nổi bọt)



procedure buble sort ($a_1,a_2,\ldots,a_n$:real numbers with $n \geq 2$)

for i:= 1 **to** n-1

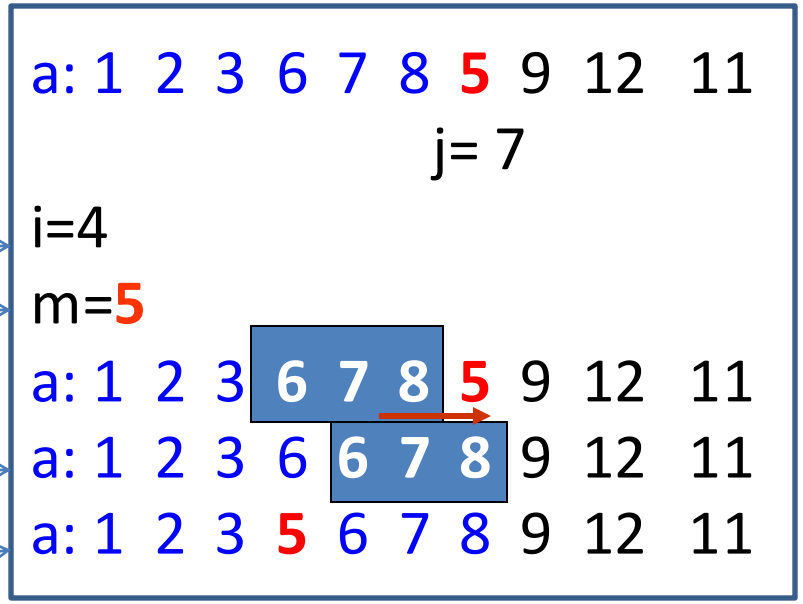
  for j:=1 **to** n- i

    if $a_j > a_{j+1}$ **then interchange** a_j **and** a_{j+1}

{$a_1,a_2,\ldots,a_n$ are sorted}

Insertion Sort (chèn)

procedure insertion sort ($a_1,a_2,\ldots,a_n$:real numbers with $n \geq 2$)
for j:= 2 **to** n {j: position of the examined element}
begin
{finding out the right position of a_j}
i:=1
while $a_j > a_i$ i:= i+1
m:=a_j {save a_j}
{ moving j-i elements backward }
for k:=0 to j-i-1 $a_{j-k}:=a_{j-k-1}$
{move a_j to the position i}
a_i:= m
end
{$a_1,a_2,\ldots,a_n$ are sorted}



It is usually not the most efficient

Greedy Algorithm – Giải thuật tham

- They are usually used to solve optimization problems: Finding out a solution to the given problem that either minimizes or maximizes the value of some parameter.
- Selecting the best choice at each step, instead of considering all sequences of steps that may lead to an optimal solution.
- Some problems:
  - Finding a route between two cities with smallest total mileage (number of miles that a person passed).
  - Determining a way to encode messages using the fewest bits possible.
  - Finding a set of fiber links between network nodes using the least amount of fiber.

Greedy Change-Making Algorithm

Procedure *change* (c_1, c_2, …, c_r: values of denominations of coins, where $c_1 > c_2 > \ldots > c_r$; n: positive integer)

for $i:=1$ **to** r

$d_i:=0$ {d_i counts the coins of denomination c_i used}

while $n \geq c_i$

$d_i:=d_i+1$ {add the coins of denomination c_i}

$n:=n-c_i$

{d_i is the number of coins of denomination c_i in the change for $i = 1, 2, \ldots, r$}

Greedy Change-Making Algorithm

Procedure *schedule* ($s_1 \le s_2 \le \ldots \le s_n$: start times of talks, $e_1 \le e_2 \le \ldots \le e_n$: ending times of talks)

sort talks by finish time and reorder so that $e_1 \le e_2 \le \ldots \le e_n$

S:=Ø

for *j:=1* **to** *n*

if talk j is compatible with S ***then***

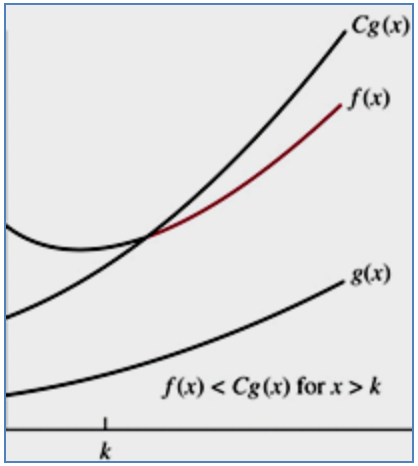
S:=SU(talk j)

return S {S is the set of talks scheduled}

3.2- The Growth of Functions

- The complexity of an algorithm that acts on a sequence depends on the number of elements of sequence.
- The growth of a function is an approach that help selecting the right algorithm to solve a problem among some of them.
- Big-O notation is a mathematical representation of the growth of a function.

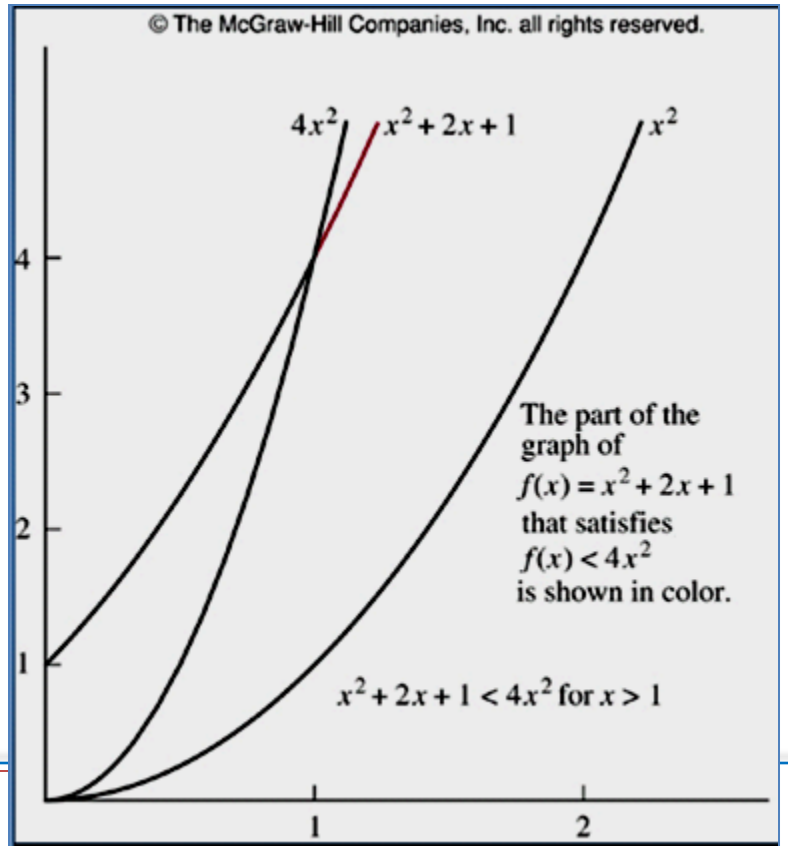
3.2.1-Big-O Notation



Definition: Let *f* and g be functions from the set of integers or the set of real numbers to the set of real numbers. We say that f(x) is O(g(x)) if there are constants C and k such that $|f(x)| \leq C|g(x)|$ whenever x>k

Example: Show that $f(x)=x^2 + 2x +1$ is $O(x^2)$

- Examine with x>1 ➔ $x^2 > x$
- ➔ $f(x)=x^2 + 2x +1 < x^2 + 2x^2 + x^2$
- ➔ $f(x) < 4x^2$
- ➔ Let $g(x)= x^2$
- ➔ C=4, k=1, $|f(x)| \leq C|g(x)|$
- ➔f(x) is $O(x^2)$



Big-O: Theorem 1

Let $f(x)=a_nx^n + a_{n-1}x^{n-1}+\ldots+a_1x+a_0$, where $a_0,a_1,\ldots,a_n$ are real number, then $f(x)$ is $O(x^n)$

If $x>1$

$|f(x)| = | a_nx^n + a_{n-1}x^{n-1}+\ldots+a_1x+a_0|$

$\leq | a_nx^n |+|a_{n-1}x^{n-1}|+\ldots+|a_1x|+|a_0|$ { triangle inequality }

$= x^n (| a_n |+|a_{n-1}x^{n-1}/x^n|+\ldots+|a_1x/x^n|+|a_0/x^n|)$

$= x^n (| a_n |+|a_{n-1}/x|+\ldots+|a_1/x^{n-1}|+|a_0/x^n|)$

$\leq x^n (| a_n |+|a_{n-1}|+\ldots+|a_1|+|a_0|)$

Let $C= |a_n|+|a_{n-1}|+\ldots+|a_1|+|a_0|$

$|f(x)| \leq Cx^n$

➔ $\mathbf{f(x) = O(x^n)}$

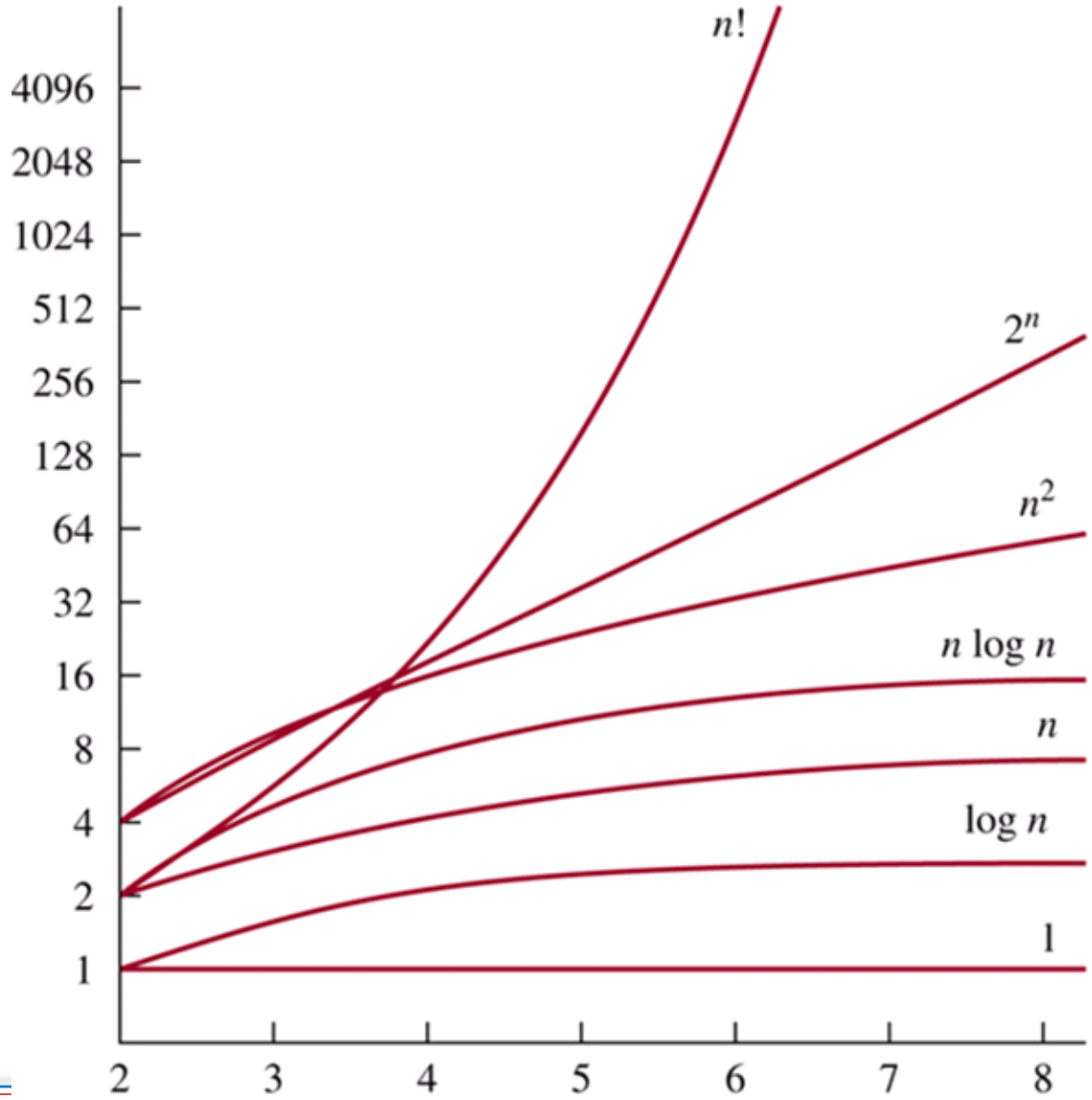
Example:

1) How can big-*O* notation be used to estimate the sum of the first *n* positive integers?

2) Give big-*O* estimates for the factorial function and the logarithm of the factorial function.

The Growth of Combinations of Functions

© The McGraw-Hill Companies, Inc. all rights reserved.



Big-O : Theorems

Theorem 2:

$f_1(x)=O(g_1(x))$ and $f_2(x)=O(g_2(x))$

$\rightarrow (f_1+f_2)(x) = O(\max(|g_1(x)|,|g_2(x)|))$

Theorem 3:

$f_1(x)=O(g_1(x))$ and $f_2(x)=O(g_2(x))$

$\rightarrow (f_1f_2)(x) = O(g_1g_2(x)))$

Corollary (hệ quả) 1:

$f_1(x)=O(g(x))$ and $f_2(x)=O(g(x)) \rightarrow (f_1+f_2)(x) = O(g(x))$

Example

Give a big-O estimate for $f(n) = 3n \log(n!) + (n^2 + 3) \log n$, where n is a positive integer.

Example

Solution: First, the product $3n \log(n!)$ will be estimated. We know that $\log(n!)$ is $O(n \log n)$ and $3n$ is $O(n)$, Theorem 3 gives the estimate that

$$3n \log(n!) \text{ is } O(n^2 \log n).$$

Next, the product $(n^2 + 3) \log n$ will be estimated.

We have, $n^2 + 3$ is $O(n^2)$.

$\Rightarrow (n^2+3)\log n$ is $O(n^2 \log n)$.

Using Theorem 2 to combine the two big-O estimates for the products shows that

$f(n) = 3n \log(n!) + (n^2+3) \log n$ *is* $O(n^2 \log n)$.

Example

Give a big-O estimate for $f(x) = (x + 1)\log(x^2 + 1) + 3x^2$.

Solution:

First, a big-O estimate for $(x + 1)\log(x^2 + 1)$ will be found. Note that $(x + 1)$ is $O(x)$. Furthermore, $x^2 + 1 \leq 2x^2$ when $x > 1$. Hence, $\log(x^2 + 1) \leq \log(2x^2) = \log 2 + \log x^2 = \log 2 + 2 \log x \leq 3 \log x$, if $x > 2$.

This shows that $\log(x^2 + 1)$ is $O(\log x)$.

From Theorem 3 it follows that $(x + 1)\log(x^2 + 1)$ is $O(x \log x)$. Because $3x^2$ is $O(x^2)$, Theorem 2 tells us that $f(x)$ is $O(\max(x \log x, x^2))$. Because $x \log x \leq x^2$, for $x > 1$, it follows that $f(x)$ is $O(x^2)$.

Exercises

1. Determine whether each of these functions is $O(x)$.

a) $f(x) = 10$
b) $f(x) = 3x + 7$
c) $f(x) = x^2 + x + 1$
d) $f(x) = 5 \log x$
e) $f(x) = \lfloor x \rfloor$
f) $f(x) = \lceil x/2 \rceil$

2. Determine whether each of these functions is $O(x^2)$.

a) $f(x) = 17x + 11$
b) $f(x) = x^2 + 1000$
c) $f(x) = x \log x$
d) $f(x) = x^4/2$
e) $f(x) = 2^x$
f) $f(x) = \lfloor x \rfloor \cdot \lceil x \rceil$

3.2.2- Big-Omega and Big-Theta Notation

- Big-O does not provide the lower bound for the size of f(x)
- Big-Ω, Big- θ were introduced by Donald Knuth in the 1970s
- Big-Ω provides the lower bound for the size of f(x)
- Big- θ provides the upper bound and lower bound on the size of f(x)

Big-Omega and Big-Theta Notation

- Definitions

$\exists c>0, x \geq k$ and $|f(x)| \geq C|(g(x)| \rightarrow |f(x)| = \Omega(g(x))$

$f(x)= O(g(x))$ and $f(x)= \Omega(g(x)) \rightarrow f(x) = \theta(g(x))$

If $f(x) = \theta(g(x))$ then $f(x)$ is of order (có bậc/loại là) $g(x)$

Show that $f(n)=1+2+\ldots+n$ is $\theta(n^2)$
Examining $n>0$
$f(n)=1+2+\ldots+n = n(n+1)/2 = (n^2+n)/2$
$f(n) \leq (2n^2)/2$
$f(n) \leq n^2$
➔Let $c_1=1/2$, $c_2=1$, $g(n)= n^2$
➔ $c_1g(n) \leq f(n) \leq c_2g(n)$
➔ $f(n) = \theta(n^2)$ with $x>0$

Example

Show that $3x^2 + 8x\log x$ is $\theta\left(x^2\right)$

Solution:

Because $0 \le 8x \log x \le 8x^2$, it follows that

$3x^2 + 8x\log x \le 11x^2$ for $x > 1$.

Consequently, $3x^2 + 8x \log x$ is $O(x^2)$. Clearly, x^2 is $O(3x^2+8x\log x)$. Consequently, $3x^2 + 8x \log x$ is $\theta\left(x^2\right)$.

Exercises

28. For each function in Exercise 1, determine whether that function is $\Omega(x)$ and whether it is $\Theta(x)$.

1. Determine whether each of these functions is $O(x)$.

a) $f(x) = 10$
b) $f(x) = 3x + 7$
c) $f(x) = x^2 + x + 1$
d) $f(x) = 5 \log x$
e) $f(x) = \lfloor x \rfloor$
f) $f(x) = \lceil x/2 \rceil$

2. Determine whether each of these functions is $O(x^2)$.

a) $f(x) = 17x + 11$
b) $f(x) = x^2 + 1000$
c) $f(x) = x \log x$
d) $f(x) = x^4/2$
e) $f(x) = 2^x$
f) $f(x) = \lfloor x \rfloor \cdot \lceil x \rceil$

3.3- Complexity of Algorithms

- Computational complexity = Time complexity + space complexity.
- Time complexity can be expressed in terms of the number of operations used by the algorithm.
  - Average-case complexity
  - Worst-case complexity
- Space complexity will not be considered.

Demo 1

Describe the time complexity of the algorithm for finding the largest element in a set:

Procedure max ($a_1, a_2, \ldots, a_n$: integers)

max:= a_1

for i:=2 to n

If max < a_i **then** max:= a_i

i	Number of comparisons
2	2
3	2
…	2
n	2
n+1	1, max< a_i is omitted

2(n-1) +1 = 2n-1 comparisions

Time Complexity is θ(n)

Demo 2

Describe the average-case time complexity of the linear-search algorithm :

Procedure linear search (x: integer, $a_1,a_2,\ldots,a_n$:distinct integers)

i:=1

while ($i \leq n$ **and** $x \neq a_i$) i:=i+1

if $i \leq n$ **then** location:= i

else location:=0

i	Number of comparisons done
1	2
2	4
…	
n	2n
n+1	1, $x \neq a_i$ is omitted

Avg-Complexity= [(2+4+6+…+2n)]/n +1
= [2(1+2+3+…+n) /n+1
= [2n(n+1)/2]/n + 1
=[n(n+1)]/n + 1
= n+1 + 1 = n+2
= θ(n)

See demonstrations about the worst-case complexity: Examples 5,6 pages 195, 196

Understanding the Complexity of Algorithms

Complexity	Terminology
$\Theta(1)$	Constant complexity
$\Theta(\log n)$	Logarithmic complexity
$\Theta(n)$	Linear complexity
$\Theta(n \log n)$	n log n complexity
$\Theta(n^b)$, b≥1, integer	Polynominal complexity
$\Theta(b^n)$, b>1	Exponential complexity
$\Theta(n!)$	Factorial complexity

10. How much time does an algorithm take to solve a problem of size n if this algorithm uses $2n^2 + 2^n$ bit operations, each requiring 10^{-9} second, with these values of n?

a) 10 **b)** 20 **c)** 50 **d)** 100

3.4- The Integers and Division

Definition: If a and b are integers with $a \neq 0$, we say that a divides b if there is an integer c such that b=ac.

When a divides b, we say that:

a is a factor (thừa số, divisor – số chia, ước số) of b

b is a multiple (bội số) of a

Notation: a|b : a divides b ałb : a does not divide b

Example:

3ł7 because 7/3 is not an integer

3|12 because 12/3=4 , remainder=0

Corollary 1:

a|b and a|c → a|(mb+nc), m,n are integers

The Division Algorithm

Theorem 2: Division Algorithm: Let a be an integer and d a positive integer. Then there are unique integers q and r, with $0 \leq r < d$, such that $a = dq + r$

d: divisor , r : remainder, q: quotient (thương số)

Note: r can not be negative

Definition: $a = dq + r$

a: dividend d: divisor

q: quotient (thương số) r : remainder,

q= a div d r = a mod d

Example: 101 is divided by 11 :101 = 11.9 + 2 ➔ q=9, r=2

-11 is divided by 3 : 3(-4)+ 1 ➔ q=-4, r=1

No OK : -11 is divided by 3 : 3(-3)-2 ➔ q=-3, **r = -2**

The Division Algorithm

23. Find a **div** m and a **mod** m when

a) $a = 228$, $m = 119$.

b) $a = 9009$, $m = 223$.

c) $a = -10101$, $m = 333$.

d) $a = -765432$, $m = 38271$.

(a) 1, 109
(b) 40, 89
(c) -31, 222
(d) -21, 38259

Modular Arithmetic

Definition: a, b: integers, m: positive integer.

a is called ***congruent*** (đồng dư) to b modulo m if m|a-b

Notations:

$a \equiv b \pmod{m}$, a is congruent to b modulo m

$a \neq b \pmod{m}$, a is not congruent to b mod m

Examples:

15 is congruent to 6 modulo 3 because 3|15-6

15 is **not** congruent to 7 modulo 3 because 3 ł15-7

Modular Arithmetic – Phép toán đồng dư

Theorem 3

a,b: integers, m: positive integer

$a \equiv b \pmod m \leftrightarrow a \bmod m = b \bmod m$

Proof

(1) $a \equiv b \pmod m \rightarrow a \bmod m = b \bmod m$

$a \equiv b \pmod m \rightarrow m \mid a-b \rightarrow a-b = km \rightarrow a = b + km$

$\rightarrow a \bmod m = (b + km) \bmod m$

$\rightarrow a \bmod m = b \bmod m \ \{ km \bmod m = 0 \}$

(2) $a \bmod m = b \bmod m \rightarrow a \equiv b \pmod m$

$a = k_1 m + c$ and $b = k_2 m + c \rightarrow a-b = (k_1-k_2)\, m$ { suppose a>b)

$= km \ \{ k = k_1-k_2 \}$

$\rightarrow m \mid a-b \rightarrow a \equiv b \pmod m$

Modular Arithmetic…

Theorem 4

a,b: integers, m: positive integer

a and b are congruent modulo m if and only if there is an integer k such that a = b + km

Proof

(1) $a \equiv b \pmod m \rightarrow a = b + km$

$a \equiv b \pmod m \rightarrow m \mid a-b \rightarrow a-b = km$ { from definition of division}

(2) $a = b + km \rightarrow a \equiv b \pmod m$

$a = b + km \rightarrow a-b=km \rightarrow m \mid a-b \rightarrow a \equiv b \pmod m$

Modular Arithmetic…

Theorem 5

m: positive integer

$a \equiv b \pmod m$ and $c \equiv d \pmod m$

$\rightarrow$ $a+c \equiv b+d \pmod m$ and $ac \equiv bd \pmod m$

Proof : See page 242

Corollary 2:

m : positive integer, a,b : integers

$(a+b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$

$ab \bmod m = ((a \bmod m)(b \bmod m)) \bmod m$

Proof: page 242

EXAMPLE 6: Because $7 \equiv 2 \pmod 5$ and $11 \equiv 1 \pmod 5$, it follows from Theorem 5 that

$$18 = 7 + 11 \equiv 2 + 1 = 3 \pmod 5$$

and that

$$77 = 7 \cdot 11 \equiv 2 \cdot 1 = 2 \pmod 5.$$

Exercises

30. Find each of these values.

a) $(177 \bmod 31 + 270 \bmod 31) \bmod 31$

b) $(177 \bmod 31 \cdot 270 \bmod 31) \bmod 31$

31. Find each of these values.

a) $(-133 \bmod 23 + 261 \bmod 23) \bmod 23$

b) $(457 \bmod 23 \cdot 182 \bmod 23) \bmod 23$

32. Find each of these values.

a) $(19^2 \bmod 41) \bmod 9$

b) $(32^3 \bmod 13)^2 \bmod 11$

c) $(7^3 \bmod 23)^2 \bmod 31$

d) $(21^2 \bmod 15)^3 \bmod 22$

30. (a) 13
(b) 19

31. (a) 13
(b) 6

32. (a) 6
(b) 9
(c) 7
(d) 18

Applications of Congruences

Hashing Function: $H(k) = k \bmod m$

Using in searching data tin memory.

k: data searched, m : memory block

Examples:

$H(064212848) \bmod 111 = 14$

$H(037149212) \bmod 111 = 65$

Collision: $H(k_1) = H(k_2)$. For example, $H(107405723) = 14$

Applications of Congruences

Pseudo-random Numbers $x_{n+1}=(ax_n+c) \bmod m$

a: multiplier, c: increment, x_0: seed

with $2 \leq a<m$, $0 \leq c<m$, $0 \leq x_0<m$

Examples:

m=9 ➔ random numbers: 0..8

a= 7, c=4, x_0=3

Result:

3, 7, 8, 6, 1, 2, 0, 4, 5, 3, 7, 8, 6, 1, 2, 0, …

Applications of Congruences

Cryptography: letter 1 → letter 2

Examples: shift cipher with k – mật mã dịch k ký tự

$f(p) = (p+k) \bmod 26 \rightarrow f^{-1}(p) = (p-k) \bmod 26$

Sender: (encoding)

Message: "ABC" , k=3

Using $f(p) = (p+3) \bmod 26$ // 26 characters

ABC ➔ 0 1 2 → 3 4 5 ➔ DEF

Receiver: (decoding)

DEF ➔ 3 4 5

Using $f^{-1}(p) = (p-3) \bmod 26$

3 4 5 ➔ 0 1 2 ➔ ABC

31. Encrypt the message "DO NOT PASS GO" by translating the letters into numbers, applying the encryption function given, and then translating the numbers back into letters.

a) $f(p) = (p + 3) \bmod 26$ (the Caesar cipher)
b) $f(p) = (p + 13) \bmod 26$
c) $f(p) = (3p + 7) \bmod 26$

32. Decrypt these messages encrypted using the Caesar cipher.

a) EOXH MHDQV
b) WHVW WRGDB
c) HDW GLP VXP

31.

A) GR QRW SDVV JR
B) QB ABG CNFF TB
C) QX UXM AHJJ ZX

32.

(a) BLUE JEANS
(b) TEST TODAY
(c) EAT DIM SUM

3.5- Primes and Greatest Common Divisors

Definition 1:

A positive integer p greater than 1 is called ***prime*** *(số nguyên tố)* if the only positive factors are 1 and p

A positive integer that is greater than 1 and is *not prime* is called *composite (hợp số)*

Examples:

Primes: 2 3 5 7 11

Composites: 4 6 8 9

Finding very large prime: test for supercomputer

Primes…

Theorem 1- The fundamental theorem of arithmetic:

Every positive integer greater than 1 can be written uniquely as a prime or as the product of two or more primes where the prime factors are written in order of nondecreasing size

Examples:

Primes: 37

Composite: $100 = 2.2.5.5 = 2^2 5^2$

$999 = 3.3.3.37 = 3^3 37$

Primes…

Converting a number to prime factors

Examples: 7007

Try it to 2,3,5 : 7007 can not divided by 2,3,5

7007 : 7

1001 : 7

143: 11

13: 13

0

➔ $7007 = 7^2.11.13$

3. Find the prime factorization of each of these integers.

a) 88 **b)** 126 **c)** 729

d) 1001 **e)** 1111 **f)** 909,090

4. Find the prime factorization of each of these integers.

a) 39 **b)** 81 **c)** 101

d) 143 **e)** 289 **f)** 899

Primes…

Theorem 2: If n is a composite, then n has a prime **divisor** less than or equal to $\sqrt{n}$

Primes…

Show that 101 is prime.

Solution:

The only primes not exceeding √101 are 2, 3, 5, and 7. Because 101 is not divisible by 2, 3, 5, or 7 (the quotient of 101 and each of these integers is not an integer), it follows that 101 is prime.

Primes…

Theorem 3: There are infinite many primes

Theorem 4: The prime number theorem:

The ratio of the number of primes not exceeding *x* and *x/lnx* approaches 1 as *x* grows with bound (ln x: natural logarithm of x)

Greatest Common Divisors and Least Common Multiples

Definition 2:

Let a, b be integers, not both zero. The largest integer d such that d|a and d|b is called the *greatest common divisor* of a and b.

Notation: gcd(a,b)

Example: gcd(24,36)=?

Divisors of 24: 2 3 4 6 8 12 = $2^3 3^1$

Divisors of 36: 2 3 4 6 9 12 18 = $2^2 3^2$

gcd(24,36)=12 = $2^2 3^1$ // Get factors having minimum power

Greatest Common Divisors and Least Common Multiples

Definition 3:

The integers a, b are *relatively prime (nguyên tố cùng nhau)* if their greatest common divisor is 1

Example:

gcd(3,7)=1 ➔ 3,7 are relatively prime

gcd (17,22)=1 ➔ 17,22 are relatively prime

gcd(17,34) = 17 ➔ 17, 34 are **not** relatively prime

Greatest Common Divisors and Least Common Multiples

Definition 4:

The integers $a_1,a_2,a_3,\ldots,a_n$ are *pairwise relatively prime* if $\gcd(a_i,a_j)=1$ whenever $1 \le i<j \le n$

Example:

7 10 11 17 23 are pairwise relatively prime

7 10 11 16 24 are **not** pairwise relatively prime

➔ Adjacent number of every composite in equence must be a prime.

Greatest Common Divisors and Least Common Multiples

Definition 5:

The Least common multiple of the positive integer a and b is the smallest integer that is divisible by both a and b

Notation: lcm(a,b)

Example:

lcm(12,36) = 36 lcm(7,11) = 77

lcm $(2^3 3^5 7^2, 2^4 3^3) = 2^4 3^5 7^2$

$2^3 3^5 7^2, 2^4 3^3 7^0 \rightarrow 2^4 3^5 7^2$ // get maximum power

Greatest Common Divisors and Least Common Multiples

Theorem 5:

Let a, b be positive integers then

ab= gcd(a,b). lcm(a,b)

Example: gcd(8, 12) = 4 lcm(8,12)=24 ➔ 8.12 = 4.24

Proof: Based on analyzing a, b to prime factors to get gcd(a,b) and lcm(a,b)

➔ ab=gcd(a,b). lcm(a,b)

24. What are the greatest common divisors of these pairs of integers?

a) $2^2 \cdot 3^3 \cdot 5^5, 2^5 \cdot 3^3 \cdot 5^2$

b) $2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13, 2^{11} \cdot 3^9 \cdot 11 \cdot 17^{14}$

c) $17, 17^{17}$

d) $2^2 \cdot 7, 5^3 \cdot 13$

e) $0, 5$

f) $2 \cdot 3 \cdot 5 \cdot 7, 2 \cdot 3 \cdot 5 \cdot 7$

30. If the product of two integers is $2^7 3^8 5^2 7^{11}$ and their greatest common divisor is $2^3 3^4 5$, what is their least common multiple?

25. What are the greatest common divisors of these pairs of integers?

a) $3^7 \cdot 5^3 \cdot 7^3, 2^{11} \cdot 3^5 \cdot 5^9$

b) $11 \cdot 13 \cdot 17, 2^9 \cdot 3^7 \cdot 5^5 \cdot 7^3$

c) $23^{31}, 23^{17}$

d) $41 \cdot 43 \cdot 53, 41 \cdot 43 \cdot 53$

e) $3^{13} \cdot 5^{17}, 2^{12} \cdot 7^{21}$

f) $1111, 0$

30. If the product of two integers is $2^7 3^8 5^2 7^{11}$ and their greatest common divisor is $2^3 3^4 5$, what is their least common multiple?

16. Determine whether the integers in each of these sets are pairwise relatively prime.

a) 21, 34, 55
b) 14, 17, 85
c) 25, 41, 49, 64
d) 17, 18, 19, 23

17. Determine whether the integers in each of these sets are pairwise relatively prime.

a) 11, 15, 19
b) 14, 15, 21
c) 12, 17, 31, 37
d) 7, 8, 9, 11

3.6- Integers and Algorithms

- Representations of Integers
- Algorithms for Integer Operations
- Modular Exponentiation
- Euclid Algorithm

Representations of Integers

Theorem 1:

Let b be a positive integer greater than 1. Then if n is a positive integer, it can be expressed uniquely in the form

$n = a_k b^k + a_{k-1} b^{k-1} + \ldots + a_1 b + a_0$

Where k is a nonnegative integer, $a_0, a_1, a_2, \ldots, a_k$ are nonnegative integers less than b and $a_k \neq 0$

Proof: page 219

Example: $(245)_8 = 2.8^2 + 4.8 + 5 = 165$

Common Bases Expansions: Binary, Octal, Decimal, Hexadecimal

4. Convert the binary expansion of each of these integers to a decimal expansion.

a) $(1\ 1011)_2$ **b)** $(10\ 1011\ 0101)_2$

c) $(11\ 1011\ 1110)_2$ **d)** $(111\ 1100\ 0001\ 1111)_2$

5. Convert the octal expansion of each of these integers to a binary expansion.

a) $(572)_8$ **b)** $(1604)_8$

c) $(423)_8$ **d)** $(2417)_8$

a) 1 01111010 **b)** 11 1000 0100

c) 1 0001 0011 **d)** 101 0000 1111

Algorithm 1: Constructing Base b Expansions

Procedure base b expansion (n: positive integer)

q:=n

k:=0

while $q \neq 0$

begin

$a_k := q \bmod b$

$q := \lfloor q/b \rfloor$

k := k +1

end { The base b expansion of n is $(a_{k-1}a_{k-2}\ldots a_1a_0)$}

Algorithms for Integer Operations

Algorithm 2: Addition integers in binary format

Algorithm 3: Multiplying integers in binary format

Algorithm 4: Computing div and mod integers

Algorithm 5: Modular Exponentiation

Algorithm 2: Adding of Integers

Complexity: O (n)

```
1 1 1 0 0
  1 1 1 0 (a)
+ 1 0 1 1 (b)
1 1 0 0 1 (s)
```

procedure add (a,b: integers)

{ a: $(a_{n-1}a_{n-2}\ldots a_1a_0)_2$ b: $(b_{n-1}b_{n-2}\ldots b_1b_0)_2$ }

c:=0

for j:=0 to n-1

Begin

d:= $\lfloor (a_j+b_j+c)/2 \rfloor$ // next carry of next step

$s_j = a_j+b_j+c - 2d$ // result bit

c:=d // updating new carry to next step

End

s_n= c // rightmost bit of result

{ The binary of expansion of the sum is $(s_ns_{n-1}\ldots s_1s_0)$}

Algorithm 3: Multiplying Integers

Complexity: $O(n^2)$

Complexity: $O(n)$

```
      1 1 0 (a)
X     1 0 1 (b)
      1 1 0
+   0 0 0 0
  1 1 0 0 0
  1 1 1 1 0 (p)
```

```
procedure multiply ( a,b: integer)
{ a: (a_{n-1}a_{n-2}…a_1a_0)_2      b: (b_{n-1}b_{n-2}…b_1b_0)_2 }
for j:= 0 to n-1
begin
   if b_j =1 then c_j := a shifted j places
end
{ c_0, c_1, …, c_{n-1}  are the partial products }
p := 0
for j:= 0 to n-1
     p:=p+c_j
{p is the value of ab}
```

Algorithm 4: Computing div and mod

```
procedure division ( a: integer; d: positive integer)
 q:=0
 r:= |a|
while r ≥ d  {quotient= number of times  of successive subtractions}
   begin
      r:= r-d
      q := q+1
   end
If a<0 and r>0  then  {updating remainder when a<0}
   begin
      r:= d-r
     q := -(q+1)
   end
{ q = a div d is the quotient, r=a mod d is the remainder}
```

Complexity: $O(n^2)$

Algorithm 5: Modular Exponentiation

{ $b^n \bmod m = ?$. Ex: $3^{644} \bmod 645 = 36$ }

procedure mod_ex (b: integer, $n=(a_{k-1}a_{k-2}\ldots a_1a_0)_2$, m: positive integer)

x:=1

power := b mod m

for i:=0 to k-1

begin

if a_i=1 then x:= (x.power) mod m

power := (power.power) mod m

end

{ x equals b^n mod m }

$$b^n = b^{a_{k-1}2^{k-1}+a_{k-2}2^{k-2}\ldots a_1 2^1+a_0}$$

$$b^n = b^{a_{k-1}2^{k-1}} . b^{a_{k-2}2^{k-2}} \ldots b^{a_1 2^1} b^{a_0}$$

$$\text{Complexity} : O\left(\log^2 m \log n\right)$$

Corollary 2: ab mod m = ((a mod m)(b mod m)) mod m

b^n mod m = result of successive factor_i mod m

Algorithm 5: Modular Exponentiation

$i = 0$: Because $a_0 = 0$, we have $x = 1$ and $power = 3^2 \bmod 645 = 9 \bmod 645 = 9$;

$i = 1$: Because $a_1 = 0$, we have $x = 1$ and $power = 9^2 \bmod 645 = 81 \bmod 645 = 81$;

$i = 2$: Because $a_2 = 1$, we have $x = 1 \cdot 81 \bmod 645 = 81$ and $power = 81^2 \bmod 645 = 6561 \bmod 645 = 111$;

$i = 3$: Because $a_3 = 0$, we have $x = 81$ and $power = 111^2 \bmod 645 = 12{,}321 \bmod 645 = 66$;

$i = 4$: Because $a_4 = 0$, we have $x = 81$ and $power = 66^2 \bmod 645 = 4356 \bmod 645 = 486$;

$i = 5$: Because $a_5 = 0$, we have $x = 81$ and $power = 486^2 \bmod 645 = 236{,}196 \bmod 645 = 126$;

$i = 6$: Because $a_6 = 0$, we have $x = 81$ and $power = 126^2 \bmod 645 = 15{,}876 \bmod 645 = 396$;

$i = 7$: Because $a_7 = 1$, we find that $x = (81 \cdot 396) \bmod 645 = 471$ and $power = 396^2 \bmod 645 = 156{,}816 \bmod 645 = 81$;

$i = 8$: Because $a_8 = 0$, we have $x = 471$ and $power = 81^2 \bmod 645 = 6561 \bmod 645 = 111$;

$i = 9$: Because $a_9 = 1$, we find that $x = (471 \cdot 111) \bmod 645 = 36$.

The Euclidean Algorithm

Lemma (bổ đề): Proof: page 228

Let a= bq+r, where a, b, q, r are integers. Then gcd(a,b) = gcd(b,r)

Example: 287 = 91.3 + 14 ➔gcd(287,91) =gcd(91,14)= 7

```
procedure  gcd(a,b: positive integer)
 x:=a
 y:=b
 while y ≠ 0
  begin
    r := x mod y
    x:=y
    y:= r
   end  {gcd(a,b) is x}
```

Summary

- Algorithms
- The Growth of Functions
- Complexity of Algorithms
- The Integers and Division
- Primes and Greatest Common Divisors
- Integers and Algorithms